R1.01 - Maîtrise des Structures Répétitives (Boucles) et Invariants

Thème principal : Algorithmique Fondamentale : Boucles (Pour, Tant Que, Répéter), Variants et Invariants de Boucle

Synthèse exécutive : Ce cours aborde le pilier fondamental de la programmation structurée : l'itération. Répéter des instructions sans dupliquer le code est essentiel pour traiter des volumes de données variables. L'enjeu majeur de ce cours est de comprendre quel type de boucle choisir selon la situation (bornée vs non bornée), d'éviter l'écueil mortel de la boucle infinie grâce à la notion de 'variant', et de garantir scientifiquement la validité d'un algorithme itératif grâce à 'l'invariant de boucle'.

1. Le Pourquoi et les trois structures d'itération

En programmation, le 'copier-coller' d'instructions est une erreur de conception majeure. Si l'on doit traiter 1000 utilisateurs ou calculer la moyenne de 50 notes, recopier 1000 fois la même ligne rend le code inmaintenable et rigide. Les structures répétitives permettent de décrire un traitement une seule fois et d'indiquer à la machine combien de fois ou sous quelles conditions elle doit l'exécuter. Il existe trois structures itératives principales : 1. 'POUR' (Boucle bornée) : On initialise une variable compteur qui varie de façon automatique entre une borne inférieure et une borne supérieure avec un pas défini. 2. 'TANT QUE' (Boucle non bornée à pré-condition) : La condition de maintien est évaluée avant chaque itération. Si la condition est fausse dès le début, le corps de la boucle n'est JAMAIS exécuté (0 fois minimum). 3. 'RÉPÉTER... JUSQU'À' (Boucle non bornée à post-condition) : Le corps est exécuté une première fois avant que la condition d'arrêt ne soit évaluée. Il y a donc au moins UNE exécution garantie (1 fois minimum).

- Iterer : Exécuter un bloc d'instructions plusieurs fois sans copier-coller.
- Boucle Bornée (POUR) : Utilisée quand le nombre exact de répétitions est déterminé avant le premier tour.
- Boucle Non-Bornée (TANT QUE / RÉPÉTER) : Utilisée quand la fin dépend d'une condition dynamique évaluée au cours de l'exécution.

Cas concret enseignant : *Le professeur a utilisé l'analogie du coureur sur une piste : Si le coach dit 'Fais 10 tours de piste', c'est une boucle POUR. Si le coach dit 'Cours tant qu'il ne pleut pas', c'est une boucle TANT QUE (s'il pleut déjà, le coureur ne court même pas). Si le coach dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est une boucle RÉPÉTER... JUSQU'À (le coureur fera au moins un tour).*

2. Preuve de Terminaison : Le Variant de Boucle

Un piège classique en partiel et dans le monde professionnel est le plantage par boucle infinie. Pour prouver formellement qu'une boucle s'arrête, l'informaticien doit identifier une valeur appelée 'variant de boucle'. Un variant est une grandeur entière qui : 1. Est strictement décroissante à chaque itération (chaque tour la diminue d'au moins 1). 2. Est minorée par une constante (par exemple 0). Puisqu'une suite d'entiers strictement décroissante et minorée est nécessairement finie, cela constitue une preuve rigoureuse que le programme ne restera pas bloqué indéfiniment.

- Boucle infinie : Plante le programme ou bloque le processeur en ne s'arrêtant jamais.
- Variant de boucle : Une expression entière strictement décroissante (ou croissante vers une borne) à chaque tour de boucle.
- Condition de terminaison : Prouve mathématiquement que la boucle va s'arrêter.

Cas concret enseignant : *L'enseignant a comparé le variant de boucle à la jauge d'essence d'une voiture : à chaque kilomètre parcouru (itération), le réservoir diminue. Comme le réservoir ne peut pas descendre en dessous de 0 litre, la voiture finira obligatoirement par s'arrêter. Si la jauge ne baisse pas, la voiture roule à l'infini (bug).*

3. Preuve de Correction : L'Invariant de Boucle

Savoir qu'une boucle s'arrête est indispensable, mais savoir qu'elle produit le BON résultat est tout aussi fondamental. C'est le rôle de l'invariant de boucle. Pour valider un invariant, on applique une démonstration similaire au raisonnement par récurrence : - Étape 1 (Initialisation) : La propriété est vérifiée avant d'entrer dans la boucle. - Étape 2 (Hérédité / Preservation) : En supposant la propriété vraie au début de l'itération k, on démontre qu'elle reste vraie à la fin de l'itération k. - Étape 3 (Conclusion) : Quand la boucle se termine, la propriété est toujours vraie, ce qui prouve que l'algorithme a calculé exactement ce qu'on lui demandait.

- Invariant de boucle : Une propriété logique qui reste VRAIE avant, pendant et après chaque itération.
- Initialisation : L'invariant est vrai avant la première itération.
- Conservation : Si l'invariant est vrai au début d'une itération, il reste vrai à la fin de l'itération.
- Correction : À la sortie de la boucle, l'invariant combiné à la négation de la condition de boucle donne le résultat attendu.

Cas concret enseignant : *Le professeur a donné l'image d'un livreur de journaux : au début de sa tournée, 'Nombre de journaux distribués + Nombre de journaux restants dans le sac = Total initial'. À chaque maison servie, les deux grandeurs changent, mais leur somme demeure rigoureusement identique. C'est l'invariant.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, absolument ! La boucle TANT QUE est la structure répétitive la plus générale. Une boucle POUR n'est qu'un sucre syntaxique d'une boucle TANT QUE dans laquelle on gère manuellement l'initialisation, le test de borne et l'incrémentation du compteur. En revanche, l'inverse n'est pas vrai : on ne peut pas remplacer une boucle TANT QUE dont le nombre d'itérations est inconnu par une boucle POUR.

Question étudiant : Si j'utilise une boucle TANT QUE avec i < N, est-ce que le corps s'exécute si i est déjà égal à N au début ?
Réponse du professeur : Non. Dans une boucle TANT QUE, la condition est évaluée A PRIORI (avant l'exécution du bloc). Si i est égal à N dès le départ, la condition i < N est fausse (FAUX), donc le programme saute directement la boucle sans jamais exécuter son contenu. C'est la différence clé avec le RÉPÉTER... JUSQU'À.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (ex: si N=3, somme de 2 + 4 + 6 = 12). Identifier clairement le variant de boucle.

Résolution pas à pas :
1. Identifier le type de boucle approprié : On connaît à l'avance le nombre d'itérations (N), une boucle POUR est donc optimale. 2. Initialiser les variables : 'somme' à 0, 'valeur_paire' à 2. 3. Rédiger le pseudo-code : Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin 4. Identification du variant de boucle : Le variant de boucle pour la terminaison est (N - i). À chaque tour, 'i' augmente de 1, donc (N - i) diminue strictement jusqu'à atteindre 0.

Résultat : Pour N = 3 : - i=1 : somme = 0 + 2 = 2, terme devient 4 - i=2 : somme = 2 + 4 = 6, terme devient 6 - i=3 : somme = 6 + 6 = 12, terme devient 8 Sortie de boucle : Résultat affiché = 12. Algorithme valide, la terminaison est garantie par le variant (N - i).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE teste sa condition AVANT d'exécuter le bloc (0 à N itérations), tandis que RÉPÉTER... JUSQU'À teste sa condition APRÈS l'exécution du bloc (au moins 1 itération garantie).

Q : À quoi sert un 'variant de boucle' lors de la conception d'un algorithme ?

R : Le variant de boucle sert à prouver la TERMINAISON de la boucle (garantir qu'elle ne tournera pas indéfiniment) en fournissant une valeur entière strictement décroissante et minorée.

Q : Quel piège classique fait perdre des points en DS lors du maniement d'une boucle TANT QUE ?

R : Oublier de modifier la variable de contrôle à l'intérieur du corps de la boucle (ex: oublier 'i <- i + 1'), ce qui rend la condition de maintien toujours vraie et provoque une boucle infinie.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765